

CALC-X

Advanced Terminal Calculator

Design Document

Program Design & Development

1. Terminology & Formulae

- **expression:** A mathematical input containing numbers, operators, parentheses, constants, and supported functions. Example: $2 + 5 * (3 + 4)$.
- **operator precedence:** The order used to evaluate an expression. CALC-X follows BODMAS: brackets, powers, multiplication/division/modulus, then addition/subtraction.
- **scientific function:** A supported mathematical function such as `sqrt()`, `cbrt()`, `log()`, `ln()`, `sin()`, `cos()`, `tan()`, `asin()`, `acos()`, and `atan()`.
- **degree mode:** Trigonometric angles are interpreted in degrees.
- **radian mode:** Trigonometric angles are interpreted in radians.
- **constant:** A predefined mathematical value such as pi or e.
- **linear equation:** An equation of the form $ax + b = 0$.
- **quadratic equation:** An equation of the form $ax^2 + bx + c = 0$.
- **discriminant:** The value used to determine the roots of a quadratic equation: $D = b^2 - 4ac$.
- **simultaneous equations:** Two or three equations containing multiple variables that are solved together to find a common solution.
- **matrix:** A rectangular arrangement of numbers represented using a two-dimensional list.
- **matrix transpose:** An operation that changes rows into columns.
- **matrix inverse:** For a square matrix A, the inverse exists only when $\det(A) \neq 0$.
- **mean:** $\text{mean} = \text{sum of values} / \text{number of values}$.
- **range:** $\text{range} = \text{maximum} - \text{minimum}$.
- **variance:** $\text{variance} = \text{sum}((x - \text{mean})^2) / n$.
- **standard deviation:** $\text{standard deviation} = \sqrt{\text{variance}}$.
- **derivative:** The rate of change of a polynomial. For ax^n , its derivative is nax^{n-1} .
- **integral:** The antiderivative of a polynomial. For ax^n , its integral is $(a/(n+1))x^{n+1} + C$.
- **history:** A list containing successfully completed calculations and their results.

Data Structures Used

Data Structure	Used For	Example
String (str)	Expressions, menu choices, function names, history entries, and polynomial input.	"2 + 5 * 3"
List (list)	Tokens, calculation history, statistical data, and polynomial coefficients.	[2, 3, 4]
Nested List (list[list])	Matrices and simultaneous-equation coefficients.	[[1, 2], [3, 4]]
Tuple (tuple)	Returning multiple calculated values such as equation roots.	(2.0, 3.0)

Data Structure	Used For	Example
Dictionary (dict)	Mapping mathematical function names to their operations.	{"sqrt": ..., "sin": ...}
Float (float)	Decimal calculation results, roots, statistics, and scientific calculations.	13.5
Integer (int)	Menu choices, matrix dimensions, counts, and exact numerical values.	9
Boolean (bool)	Validation results and condition checks.	True / False

2. Program Flow

1. Start the program.
2. Initialise an empty calculation history.
3. Set the scientific calculator mode.
4. Display the CALC-X main menu.
5. Read the user's menu choice.
6. If the choice is 1, open Basic Calculator.
7. If the choice is 2, open Scientific Calculator.
8. If the choice is 3, open Algebra Solver.
9. If the choice is 4, open Equation Solver.
10. If the choice is 5, open Matrix Operations.
11. If the choice is 6, open Statistics.
12. If the choice is 7, open Calculus.
13. If the choice is 8, open History.
14. If the choice is 9, print Goodbye! and terminate the program.
15. For any other choice, print Invalid choice. and display the menu again.
16. After completing a module, return to the main menu.

Main Menu

1. Basic Calculator
2. Scientific Calculator
3. Algebra Solver
4. Equation Solver
5. Matrix Operations
6. Statistics
7. Calculus
8. History
9. Exit

3. Function Signatures

calculator.py

Typed header line	What it's for
def validate_expression(expression: str) -> bool:	Returns True if the expression has valid basic syntax, otherwise False.
def tokenize(expression: str) -> list:	Converts an expression into numbers, operators, parentheses, constants, and functions.
def evaluate_expression(expression: str, mode: str) -> float:	Evaluates a complete mathematical expression according to BODMAS.
def apply_function(name: str, value: float, mode: str) -> float:	Calculates the requested scientific function using the selected angle mode.
def calculate_power(base: float, exponent: float) -> float:	Returns the value of a number raised to a power.
def calculate_log(value: float, base: float = 10) -> float:	Returns the logarithm of a value using the specified base.

algebra.py

Typed header line	What it's for
def solve_linear(a: float, b: float) -> float:	Solves $ax + b = 0$ and returns x .
def solve_quadratic(a: float, b: float, c: float) -> tuple:	Calculates the real roots of a quadratic equation.
def evaluate_polynomial(coefficients: list, x: float) -> float:	Evaluates a polynomial for a given value of x .
def polynomial_degree(coefficients: list) -> int:	Returns the degree of a polynomial from its coefficient list.

equations.py

Typed header line	What it's for
def solve_2x2(coefficients: list) -> tuple:	Solves a system of two equations with two variables.
def solve_3x3(coefficients: list) -> tuple:	Solves a system of three equations with three variables.
def has_unique_solution(coefficients: list) -> bool:	Checks whether the equation system has a unique solution.

matrix.py

Typed header line	What it's for
def add_matrices(A: list, B: list) -> list:	Returns the sum of two matrices with matching dimensions.
def subtract_matrices(A: list, B: list) -> list:	Returns the difference of two matrices with matching dimensions.
def multiply_matrices(A: list, B: list) -> list:	Returns the matrix product when dimensions are compatible.
def transpose_matrix(A: list) -> list:	Returns the transpose of a matrix.
def determinant(A: list) -> float:	Returns the determinant of a square matrix.
def inverse_matrix(A: list) -> list:	Returns the inverse of a non-singular square matrix.
def valid_dimensions(A: list, B: list) -> bool:	Checks whether matrix dimensions are valid for the requested operation.

statistics.py

Typed header line	What it's for
def calculate_mean(data: list) -> float:	Returns the arithmetic mean.

Typed header line	What it's for
def calculate_median(data: list) -> float:	Returns the median.
def calculate_mode(data: list) -> list:	Returns the most frequently occurring value or values.
def calculate_min(data: list) -> float:	Returns the smallest value.
def calculate_max(data: list) -> float:	Returns the largest value.
def calculate_range(data: list) -> float:	Returns maximum minus minimum.
def calculate_variance(data: list) -> float:	Returns the variance.
def calculate_standard_deviation(data: list) -> float:	Returns the standard deviation.

calculus.py

Typed header line	What it's for
def differentiate_polynomial(coefficients: list) -> list:	Returns the coefficients of the derivative of a polynomial.
def integrate_polynomial(coefficients: list) -> list:	Returns the coefficients of the basic integral of a polynomial.
def format_polynomial(coefficients: list) -> str:	Converts polynomial coefficients into readable mathematical text.

history.py

Typed header line	What it's for
def add_history(history: list, expression: str, result: str) -> None:	Adds a successful calculation to history.
def display_history(history: list) -> None:	Displays stored calculations.
def save_history(history: list, filename: str) -> None:	Saves calculation history to a file.
def load_history(filename: str) -> list:	Loads previously saved calculations from a file.
def clear_history(history: list) -> None:	Removes all stored history entries.

main.py

Typed header line	What it's for
def print_menu() -> None:	Displays the CALC-X main menu.
def get_menu_choice() -> str:	Reads the user's menu choice.
def basic_calculator(history: list) -> None:	Handles the basic calculator interaction.
def scientific_calculator(history: list) -> None:	Handles scientific calculations and angle mode.
def algebra_solver() -> None:	Handles linear, quadratic, and polynomial calculations.
def equation_solver() -> None:	Handles simultaneous equation solving.
def matrix_operations() -> None:	Handles matrix operations and displays results.
def statistics_module() -> None:	Handles statistical calculations.
def calculus_module() -> None:	Handles differentiation and integration.
def history_menu(history: list) -> None:	Displays and manages calculation history.
def main() -> None:	Runs the main CALC-X menu loop and controls program navigation.

4. Function-Level Algorithm

validate_expression(expression)

- Check whether the expression is empty.
- Check that parentheses are correctly matched.
- Check that operators and operands appear in valid positions.
- Check that supported functions and constants are used correctly.
- Return True when the expression is valid.
- Return False otherwise.

tokenize(expression)

- Read the expression from left to right.
- Identify numbers and decimal values.
- Identify mathematical operators.
- Identify parentheses.
- Identify function names such as sqrt, log, and sin.
- Identify constants such as pi and e.
- Store the identified elements in a list.
- Return the list of tokens.

evaluate_expression(expression, mode)

- Validate the expression.
- Tokenize the expression.
- Evaluate expressions inside parentheses first.
- Evaluate mathematical functions.
- Evaluate powers.
- Evaluate multiplication, division, and modulus.
- Evaluate addition and subtraction.
- Return the final result.
- If an invalid mathematical operation occurs, report the error without terminating the program.

apply_function(name, value, mode)

- Check the requested function name.
- For sqrt, calculate the square root.
- For cbrt, calculate the cube root.
- For log, calculate the base-10 logarithm.
- For ln, calculate the natural logarithm.
- For trigonometric functions, use the selected degree or radian mode.
- For abs, floor, and ceil, calculate the corresponding value.
- Return the result.

solve_linear(a, b)

- Receive a and b.

- Check whether a is zero.
- If a is not zero, calculate $x = -b / a$.
- Return the solution.
- If a is zero, report that there is no unique solution.

solve_quadratic(a, b, c)

- Calculate $D = b^2 - 4ac$.
- If $D > 0$, calculate two real roots.
- If $D == 0$, calculate one repeated root.
- If $D < 0$, report that there are no real roots.
- Return the result.

solve_2x2(coefficients)

- Receive coefficients for two simultaneous equations.
- Calculate the determinant of the coefficient system.
- Check whether the determinant is zero.
- If it is not zero, calculate x and y.
- Return the solution.
- If it is zero, report that there is no unique solution.

solve_3x3(coefficients)

- Receive coefficients for three simultaneous equations.
- Validate the coefficient matrix.
- Apply elimination to reduce the system.
- Calculate the three variable values.
- Return the solution.
- If the system does not have a unique solution, report an appropriate error.

add_matrices(A, B)

- Check that both matrices have the same dimensions.
- Add corresponding elements.
- Store the results in a new matrix.
- Return the new matrix.

subtract_matrices(A, B)

- Check that both matrices have the same dimensions.
- Subtract corresponding elements.
- Store the results in a new matrix.
- Return the new matrix.

multiply_matrices(A, B)

- Check that the number of columns in A equals the number of rows in B.
- Multiply each row of A by each column of B.
- Add the products for each result position.

- Return the resulting matrix.

transpose_matrix(A)

- Read the columns of the matrix.
- Convert each column into a row.
- Return the resulting matrix.

determinant(A)

- Check that the matrix is square.
- For a 2×2 matrix, use $ad - bc$.
- For larger supported matrices, calculate the determinant using expansion or elimination.
- Return the determinant.

inverse_matrix(A)

- Check that the matrix is square.
- Calculate its determinant.
- If the determinant is zero, report that the inverse does not exist.
- Calculate the inverse.
- Return the resulting matrix.

calculate_mean(data)

- Add all values in the dataset.
- Count the number of values.
- Divide the total by the count.
- Return the mean.

calculate_median(data)

- Sort the dataset.
- Find the middle value.
- If the number of values is even, calculate the average of the two middle values.
- Return the median.

calculate_mode(data)

- Count how many times each value occurs.
- Find the highest frequency.
- Return the value or values having that frequency.

calculate_variance(data)

- Calculate the mean.
- Find the difference between each value and the mean.
- Square each difference.
- Add the squared differences.
- Divide by the number of values.
- Return the variance.

calculate_standard_deviation(data)

- Calculate the variance.
- Find the square root of the variance.
- Return the standard deviation.

differentiate_polynomial(coefficients)

- Read each polynomial coefficient and its exponent.
- Multiply the coefficient by its exponent.
- Reduce the exponent by one.
- Store the new coefficients.
- Return the derivative.

Example: $3x^2 + 4x + 5 \rightarrow 6x + 4$

integrate_polynomial(coefficients)

- Read each polynomial coefficient and its exponent.
- Increase the exponent by one.
- Divide the coefficient by the new exponent.
- Store the resulting terms.
- Add the integration constant C.
- Return the integral.

Example: $6x + 4 \rightarrow 3x^2 + 4x + C$

add_history(history, expression, result)

- Receive a successful expression and its result.
- Format them as a history entry.
- Add the entry to the history list.
- Do not add failed calculations.

save_history(history, filename)

- Open the specified history file.
- Write each history entry to the file.
- Close the file.
- Report a file error if saving is unsuccessful.

load_history(filename)

- Check whether the history file exists.
- Open the file.
- Read each stored calculation.
- Create a history list.
- Return the list.
- If the file cannot be accessed, return an appropriate error.

main()

- Initialise the history list.
- Display the CALC-X menu.

- Read the menu choice.
- Call the function corresponding to the selected module.
- After the module finishes, display the menu again.
- If the user selects 9, print Goodbye! and end the program.
- Continue until the user chooses Exit.